SEGUNDO DEPARTAMENTAL- EJERCICIO 3- PILA

En este programa desarrollamos una pila estática, el objetivo es permitir el almacenamiento, inserción, eliminación, consulta y visualización de elementos enteros, siguiendo el principio LIFO, que significa que el último elemento en entrar es el primero en salir. Para lograrlo, creamos la clase Pila, en la cual implementamos un arreglo de tamaño fijo de cinco elementos y una variable llamada tope, encargada de controlar la posición del último elemento insertado, también incorporamos un menú interactivo que nos permite realizar las operaciones principales de una pila: PUSH, POP y PEEK, además de mostrar los elementos almacenados, limpiar la pantalla y finalizar la ejecución.

Errores y Correcciones: En la versión inicial, al momento de ejecutar en repetidas ocasiones alguna opción del menú la pantalla se llenaba información lo que causaba confusión al momento de comprobar la operación solicitada. Por lo que agregamos una nueva opción al menú mediante la estructura switch: “Limpiar pantalla”.

Código Completo:

```
#include <iostream>
#include <stdlib.h>
using namespace std;

const int MAX = 5;

class Pila {
private:
    int datos[MAX];
    int tope;

public:
    // Constructor
    Pila() {
        tope = -1;
    }

    // Insertar elemento (PUSH)
    void insertar() {
        int dato;

        if (tope == MAX - 1) {
            cout << "La pila esta llena. Limite de 5 elementos.\n";
        } else {
            cout << "Ingresa un numero: ";
            cin >> dato;

            tope++;
            datos[tope] = dato;

            cout << "Elemento insertado correctamente.\n";
        }
    }

    // Eliminar elemento (POP)
    void eliminar() {
        if (tope == -1) {
            cout << "La pila esta vacia.\n";
```

```
        } else {
            cout << "Elemento eliminado: "
                 << datos[tope] << endl;
            tope--;
        }
    }

    // Mostrar elementos de la pila
    void mostrar() {
        if (tope == -1) {
            cout << "La pila esta vacia.\n";
        } else {
            cout << "\nElementos de la pila:\n";

            for (int i = tope; i >= 0; i--) {
                cout << datos[i] << endl;
            }
        }
    }

    // Consultar elemento superior (PEEK)
    void consultarTope() {
        if (tope == -1) {
            cout << "La pila esta vacia.\n";
        } else {
            cout << "Elemento en el tope: "
                 << datos[tope] << endl;
        }
    }
};

int main() {
    Pila pila;
    int opcion;

    do {
        cout << "\n=== PILA ESTATICA EN POO ===\n";
        cout << "1. Insertar elemento (PUSH)\n";
        cout << "2. Eliminar elemento (POP)\n";
        cout << "3. Mostrar pila\n";
        cout << "4. Consultar tope (PEEK)\n";
        cout << "5. Limpiar pantalla\n";
        cout << "6. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                pila.insertar();
                break;

            case 2:
                pila.eliminar();
                break;

            case 3:
                pila.mostrar();
                break;

            case 4:
                pila.consultarTope();
                break;
```

```
            case 5:
                system("cls");
                break;

            case 6:
                cout << "Programa finalizado.\n";
                break;

            default:
                cout << "Opcion no valida.\n";
        }

    } while (opcion != 6);

    return 0;
}
```

Explicación del Código:

1. Declaración de la Constante Máxima (const int MAX = 5)

Declaramos una constante llamada MAX, a la cual le asignamos el valor de 5, con esta constante establecemos la capacidad máxima de nuestra pila, al utilizar una constante, podemos modificar fácilmente el límite de almacenamiento sin tener que cambiar manualmente el tamaño del arreglo en diferentes partes del programa.

2. Declaración de la Clase Pila (class Pila)

Creamos la clase Pila, donde agrupamos los atributos y métodos necesarios para administrar nuestra estructura de datos, dentro de esta clase implementamos las operaciones de inserción, eliminación, visualización y consulta del elemento superior, de esta manera, organizamos las funciones relacionadas con la pila en una sola clase.

3. Declaración de los Atributos Privados (datos[MAX] y tope)

Dentro de la sección private declaramos los siguientes atributos:

- datos[MAX]: es un arreglo de tipo entero que utilizamos para almacenar los elementos de nuestra pila. Su capacidad máxima es de cinco números.
- tope: es una variable de tipo entero que utilizamos para indicar la posición del último elemento insertado.

4. Declaración del Constructor (Pila())

Definimos un constructor que se ejecuta automáticamente cuando creamos un objeto de la clase Pila, su función es inicializar la variable tope con el valor -1. Asignamos este valor porque representa que nuestra pila se encuentra vacía, debido a que todavía no hemos insertado ningún elemento, cuando agregamos el primer número, la variable tope aumenta a cero y el elemento se almacena en la primera posición del arreglo.

5. Método para Insertar Elementos (void insertar())

Creamos el método insertar(), cuya función es agregar un nuevo elemento a nuestra pila, esta operación también se conoce como PUSH, dentro del método declaramos la variable dato, que utilizamos para almacenar temporalmente el número entero que deseamos insertar, antes de realizar la operación, verificamos si nuestra pila todavía tiene espacio disponible.

6. Validación de Pila Llena (if (tope == MAX - 1))

Utilizamos una estructura condicional if para comprobar si nuestra pila alcanzó su capacidad máxima, la condición compara el valor de tope con MAX – 1, si se cumple esta condición, mostramos un mensaje indicando que la pila está llena y evitamos que se inserte otro elemento.

7. Captura e Inserción de un Elemento (cin >> dato)

Cuando nuestra pila todavía tiene espacio disponible, solicitamos al usuario que ingrese un número entero y lo almacenamos temporalmente en la variable dato, después, incrementamos la variable tope y almacenamos el número en la posición correspondiente:

8. Método para Eliminar Elementos (void eliminar())

Creamos el método eliminar(), cuya función es retirar el elemento que se encuentra en la parte superior de nuestra pila, esta operación también se conoce como POP, antes de eliminar un elemento, verificamos si nuestra pila contiene información, si tope tiene el valor -1, significa que no existen elementos almacenados.

9. Eliminación del Elemento Superior (tope--)

Si nuestra pila contiene elementos, primero, mostramos el valor almacenado en la posición indicada por tope, después, utilizamos la instrucción tope-- para disminuir en uno la posición superior de nuestra pila.

10. Método para Mostrar los Elementos (void mostrar())

Creamos el método mostrar(), cuya función es visualizar todos los elementos que se encuentran almacenados en nuestra pila, primero verificamos si nuestra pila está vacía mediante la condición tope == -1, si no existen elementos, mostramos un mensaje indicando que la pila está vacía, en caso contrario, presentamos el contenido de la estructura.

11. Recorrido de la Pila mediante un Ciclo (for)

Cuando nuestra pila contiene elementos, utilizamos un ciclo for para recorrer el arreglo desde la posición superior hasta la primera posición. El ciclo comienza en i = tope y disminuye su valor hasta llegar a cero, así, mostramos los elementos desde el último que insertamos hasta el primero.

12. Método para Consultar el Elemento Superior (void consultarTope())

Creamos el método consultarTope(), cuya función es mostrar el elemento que se encuentra en la parte superior de nuestra pila sin eliminarlo, esta operación también se conoce como PEEK, si tope tiene el valor -1, mostramos un mensaje indicando que no existe ningún elemento para consultar.

13. Consulta del Elemento Superior (datos[tope])

Cuando nuestra pila contiene elementos, utilizamos datos[tope] para acceder al elemento ubicado en la posición superior de nuestra pila, a diferencia de la operación POP, con PEEK únicamente mostramos el elemento y no modificamos el valor de tope.

14. Función Principal (int main())

Dentro de esta función creamos un objeto llamado pila, perteneciente a nuestra clase Pila, al crear este objeto, se ejecuta automáticamente el constructor y se inicializa tope con el valor -1, también declaramos la variable opcion, que utilizamos para almacenar la selección realizada por el usuario en el menú principal.